

Vasic-Digital-Reusable-Module-Suite

한 번 구축하고, 어디서나 재사용 — 독립 테스트된 작은 **Go** 및 **KMP** 모듈군

요약

digital.vasic.*(Go) 및 Kotlin Multiplatform 네임스페이스 하에 공개된 범용 재사용 모듈군. 각 모듈은 독립적으로 테스트되고 버전 관리되며, 더 큰 제품(Catalogizer, HelixAgent 및 기타 모듈군)에 동등한 코드베이스 하위 모듈로 통합됩니다. 이 페이지는 개별 페이지로 다루기에는 너무 작은 수많은 유틸리티를 한데 모아 정리한 것입니다.

간략 설명

독립적으로 설계된 digital.vasic.* 모듈군 — 인프라 기본 요소(인증, 캐시, 데이터베이스, 설정, 관측 가능성), AI/에이전트 구성 요소(RAG, 벡터 DB, 임베딩, MCP, 에이전틱, 플래닝), 방어적 LLM 가드레일(RedTeam, Normalize) — 그리고 Kotlin Multiplatform 미러 세트로 구성된 선별된 모듈 모음입니다. 각각은 범용적이며 테스트를 거쳤고 재사용이 가능합니다.

상세 설명

vasic-digital 조직은 하나의 구조적 원칙에 기반을 둡니다. 바로 “헌법 + 수많은 독립 재사용 모듈” 철학입니다. 범용 기능은 결코 두 번 작성되지 않습니다. 모놀리식 구조 대신, 모든 재사용 가능한 기능은 자체 저장소, 자체 테스트, 자체 문서로 분리된 작은 모듈로 추출되며, 소비자의 세부 사항이 유입되지 않도록 철저히 독립성을 유지합니다. 이 페이지는 각 모듈이 라이브러리 수준으로 작아 개별 제품 페이지로는 소음으로 느껴질 수 있기 때문에 이를 한데 묶어 정리한 것입니다. 그러나 전체적으로 보면 이들은 조직의 진정한 역량 증폭기입니다. 바로 “새로운 제품을 구축한다”를 “검증된 부품을 조립한다”로 바꾸는 사적 엔지니어링 자산이자, 이 모듈군이 바퀴를 재발명하지 않고 하나의 뛰어난 바퀴를 유지하며 이를 곳곳에 활용한다는 주장의 구체적 증거입니다.

이 모듈군은 세 가지 클러스터로 나뉩니다. 인프라 기본 요소(Go)는 모든 서비스가 필요로 하는 기반을 제공합니다: auth(JWT/bcrypt), cache(Redis/TTL), database(마이그레이션, 이중 SQLite/PostgreSQL), config, middleware, observability(Prometheus/OpenTelemetry), ratelimiter, security, storage(S3/MinIO), streaming(WebSocket 허브), eventbus, filesystem(다중 프로토콜), discovery/mdns, http3, recovery, concurrency, lazy 등. **AI/에이전트** 구성 요소(Go)는 AI 시스템을 위한 기반을 제공합니다: rag, vectordb, embeddings, memory, conversation(무한 컨텍스트 압축, 이벤트 소싱), mcp(Model Context Protocol), toolschema, skillregistry, agentic(그래프 기반 워크플로 오케스트레이션), planning(HiPlan/MCTS/Tree-of-Thoughts), benchmark(SWE-bench/HumanEval/MMLU), llmops, selfimprove(보상 모델링/RLHF), toon(Token-Oriented Object Notation). 방어적 **LLM** 가드레일은 적대적 환경에 대한 견고성을 제공합니다: RedTeam(YAML 기반 적대적 피쳐), Normalize(적대적 입력 표준화). 이와 병행하여 **Kotlin Multiplatform** 세트는 크로스 플랫폼 앱을 위한 핵심 모듈의 미러 버전을 제공합니다 (Auth-KMP, Database-KMP, Security-KMP, UI-Components-KMP 등).

콘텐츠

왜 만들었는가

매번 새로운 제품을 출시할 때마다(Catalogizer, HelixAgent, Herald 등) 모든 것을 처음부터 개발하는 것은 비효율적이고 일관성도 떨어집니다. 일반적인 문제를 독립적이고 테스트된 모듈로 분리하면 수정 사항과 개선 사항이 전체 제품군에 자동으로 적용되며, 새로운 제품도 검증된 구성 요소로 조립할 수 있습니다.

혁신적인 이유

이는 사실상 AI 중심 백엔드를 구축하기 위한 '사실 표준 라이브러리'와 같습니다. 대부분의 팀이 인증, 캐싱, RAG 관련 기반 작업에 매번 매달리느라 제대로 구축하지 못하는 레이어죠. 여기에는 인프라 기본 요소, AI 빌딩 블록, 방어형 LLM 안전장치가 모두 독립적으로 테스트된 드롭인 모듈로 존재합니다. 덕분에 소규모 팀도 훨씬 큰 조직이 필요한 속도로 프로덕션급 시스템을 출시할 수 있으며, 그 과정에서 발생하는 중복 부채도 쌓이지 않습니다.

혁신적인 점

- 전체 제품군에 걸친 디커플링 원칙(**CONST-051**): 하위 모듈을 독립적인 코드베이스로 취급하며, 소비자별 특성을 포함하지 않습니다.
- 전용 **AI** 프리미티브 레이어(**RAG, VectorDB, 임베딩, MCP, ToolSchema, 에이전틱, 플래닝, LLMops**): 재사용 가능한 모듈로 구성됩니다.

- 방어형 **LLM** 안전장치 클러스터(**RedTeam, Normalize**): 적대적 환경에서의 견고성을 확보합니다.
- 동일한 규칙을 공유하는 병렬 **Go + Kotlin Multiplatform** 모듈 세트

도전 과제와 해결 방안

- 수십 개의 모듈 간 결합 부패 방지: '헌장'의 디커플링 계약과 런타임 주입을 통한 소비자별 특성 적용으로 해결했습니다.
- 다수의 모듈 일관성 유지 및 테스트: 공통 규칙(모듈별 테스트/문서/도전 과제)과 HelixConstitution 거버넌스 백본으로 해결했습니다.
- 크로스 플랫폼 지원: 핵심 모듈의 Kotlin Multiplatform 미러링으로 해결했습니다.

기술 스택(이유와 방법)

- **Go** — 대부분의 모듈(digital.vasic.*)
- **Kotlin Multiplatform** — 크로스 플랫폼 미러 모듈(인증/데이터베이스/보안/UI/동시성/RateLimiter-KMP)
- **Redis / PostgreSQL / SQLite** — 캐시, 데이터베이스, 스토리지 프리미티브
- **Prometheus / OpenTelemetry** — 관측 가능성 모듈
- **WebSocket / HTTP/3 (quic-go) / mDNS** — 네트워킹 모듈
- **Vector DB / embeddings / RAG / MCP** — AI 프리미티브 모듈
- **YAML** — RedTeam 적대적 테스트 케이스 및 설정

미검증/작업 중: 일부 조직 저장소는 “SCAFFOLD / WIP”로 자체 표시되어 있습니다(PliniusCommon, I-LLM, HyperTune, AutoTemp, Veritas, Ouroborous, Claritas, LeakHub, GandalfSolutions). 현재 단계에서는 초기 개발/스캐폴드 상태로 제공되며, 공식 출시된 상태가 아님을 명시합니다.